

RESULTS

실험 배터리 결과 (작은 실험 전부 실행)

실행 환경: RTX 4090 24GB (martin@linux-builder), torch 2.10.0+cu128, cuda_op_ok: True . 재현: scripts/run_experiments.sh (mock 배터리) · run.py run --config config/run_real.yaml (실물).

A. 실물(real) 백엔드 — 실제 PyTorch 학습 결과 ★

growing-memory-pytorch 업스트림이 없어 설계 4축 + SSL을 torch로 직접 구현(autoresearch/real/) 하고, 실제 MQAR 연관회상 과제로 4090에서 학습/평가. backend= real .

실물 ASHA (rungs=[400,1000,2500] 실제 학습 스텝, eta=3, max_trials=9, MQAR 4쌍/vocab40):

trial	config	proxy(recall)	full	cost(step)	승급
t0006	dla+grm d256L4 seg32	0.327	0.249	2500	→ top rung
t0002	swla+residual d128L2	0.309	-	1000	→ rung1
t0005	titans+residual d256L2	0.308	-	1000	→ rung1
t0008	swla+grm d256L4	0.299	-	1000	→ rung1
t0004	linear+ssc d256L2	0.249	-	400	rung0
t0003	titans+soup	0.094	-	400	rung0(조기중단)
t0007	linear+soup	0.034	-	400	rung0(조기중단)
t0001	titans+ssc	0.000	-	400	rung0(조기중단)

chance recall $\approx$ 0.05. backend=real, jobs_run=14, gate 탈락 0.

발견 (실물)

1. **실학습 파이프라인 동작 확인.** build(cfg)→실제 torch 모델→AdamW 학습(2500스텝)→MQAR recall 평가까지 4090에서 end-to-end. best= dla+grm recall 0.249(chance 0.05의 ~5배).
2. **ASHA 조기중단이 실측으로 작동.** rung0(400스텝)에서 proxy 낮은 config(0.0~0.09)는 승급 차단, 상위만 1000→2500으로 승급. 자원 낭비 없이 학습되는 config로 집중.
3. **스wijk이 "작동 조합"을 발견 — autoresearch의 본질.** 단일 고정 config 스모크에선 dla/titans/ssc가 chance였으나, 탐색은 dla+grm · titans+residual · linear+ssc 등 학습되는 조합을 찾아냄. 조합 (aggregation×base_rule×하이퍼파라미터) 의존성이 크다는 것을 실측으로 확인.

4. **MQAR grokking 한계**. recall은 급격 상전이라 일부 조합(soup, titans+ssc)은 현재 예산에서 임계 미달. 예산 ↑/튜닝으로 보정 대상(real/README.md). 선형/소프트맥스 어텐션이 회상에 유리 (Zoology 경향)와 일치.

실물 연결: 머신에서 `export GROWING_MEMORY_HOME=<repo>` 시 업스트림 우선, 없으면 이 참조 구현, torch 없으면 mock — 인터페이스 무변경. 강제 mock: `AR_FORCE MOCK=1` .

B. mock 백엔드 — 루프 폐쇄/탐색 로직 검증

백엔드 = mock(torch 없이 stdlib, 합성 신호). 탐색·게이트·리더보드·ratchet 로직 검증용.

스윙 실험 (ASHA, rungs=[1000,4000,16000], eta=4, max_trials=24)

실험	proxy	표현축	jobs	gate 탈락	top rung 도달	rank_corr	best full	best 계열
E1	factory_mqar	full(SSL)	35	10	3	0.50	0.7812	titans+ssc
E2	short_horizon_pred	full(SSL)	37	13	3	1.00	0.7812	titans+ssc
E3	factory_mqar	T0(SSL 고정)	35	6	2	1.00	0.6018	titans+residual
E4 (seed1)	factory_mqar	full	31	4	1	-	0.7818	titans+ssc
E4 (seed2)	factory_mqar	full	32	5	1	-	0.7562	titans+ssc
E4 (seed3)	factory_mqar	full	31	4	1	-	0.7749	dla+ssc

발견

- 표현축(SSL)이 가장 큰 레버 — 실측 확인**. 풀공간 best(0.78) vs T0(SSL 고정) best(0.60), 절대 +0.18. 설계 §3.2의 "positive_pair가 가장 큰 레버" 가설과 일치.
- best 계열 수렴 안정성**. 시드 1·2·3에서 best가 모두 `*+ssc` 기억축 + `titans/dla` base로 수렴(full 0.75~0.78). 작은 모델 + 효율 RNN으로 싸게 많이 시도해도 winner 계열이 일관됨.
- 프록시 신뢰도는 과제 의존**. short_horizon_pred는 rank_corr=1.0(프록시↔full 완벽 일치), factory_mqar는 0.5(medium). → §5대로 프록시는 주기적 보정 필요.
- ASHA 한계(관측)**. max_trials=24·eta=4에서는 top rung 생존이 1~3개에 그쳐, 시드에 따라 rank_corr 계산 불가(≥2 필요). 보정안: max_trials↑ 또는 eta↓로 상위 rung 생존 수 확보.

Skill / 원형 / 인터페이스 점검

항목	결과
validity-gate (space 300 샘플)	통과율 80.7%, 탈락 전부 <code>d_model % n_heads</code> (게이트 정상)
leaderboard-analysis (E1)	rank_corr 0.5 → proxy_trust=medium, best export_safe=True
lab ratchet (karpathy 원형, snapshot 30iter)	seed0: best 1.0228 (keep 11 / revert 20), seed1: best 1.0531 (keep 9 / revert 22) — val_bpb 단조 개선
MCP selftest	7 도구 정상(<code>leaderboard_top/summary/get/write</code> , <code>nas_list_shards</code> , <code>run_status</code> , <code>export_best</code>)
export (E1 best)	번들 생성 OK, encoder=vjepa, full=0.7812

다음 단계

- `GROWING_MEMORY_HOME` 연결 시 동일 배터리가 **실학습**으로 전환(mock→real). 인터페이스 무변경.
- factory_mqar 프록시를 공장 데이터 구조에 맞춰 구체화(\$11) → rank_corr 개선.